SYSTEM ARCHITECTURE · V1.0

Modbus Simulator

Python 3.11 · FastAPI · pymodbus
React 18 · Vite · WebSocket
Single-process · No persistence

Modbus / External | Core / State | Real-time | REST API / Frontend | Config / Infra

daemon thread — Modbus runs in background thread
asyncio — FastAPI runs on event loop

▶ EXTERNAL CLIENTS

Modbus Master
Any Modbus TCP client under test — SCADA, PLC emulator, test tool
FC1–FC6, FC15, FC16

↕ TCP :502

CSV File
Register definitions. Volume-mounted or uploaded at runtime
registers.csv

on startup / reload ↓

Docker / CLI
Runtime host. Exposes network ports
:502 :8000 :3000

⬡ BACKEND — PYTHON PROCESS

STARTUP · main.py

CSV Loader
Parses & validates CSV into Register objects. Called on boot + hot-reload
core/csv_loader.py

Register Store
In-memory dict. Single source of truth. asyncio.Lock for thread safety
core/store.py

MODBUS TCP SERVER — daemon thread

TCP Slave
pymodbus StartTcpServer. Handles all Modbus function codes from external masters
modbus/server.py

Data Block
Custom ModbusSparseDataBlock. Delegates R/W to RegisterStore. Emits change events on write
modbus/datablock.py

REAL-TIME LAYER — asyncio

Event Bus asyncio.Queue
Decouples producers (Modbus writes, API PATCH) from consumers (WebSocket broadcaster). Thread-safe bridge via run_coroutine_threadsafe
core/events.py · RegisterChangeEvent

REST API + WEBSOCKET — asyncio · uvicorn

Registers
GET · POST · PATCH · PUT /meta · DELETE
routes/registers.py

Config
POST /reload · POST /upload — hot-swap CSV
routes/config.py

WebSocket
Snapshot on connect. Streams change events. Auto fan-out to all clients
api/websocket.py

:8000 REST | :8000/ws

◈ FRONTEND — REACT

StatusBar
WS health, Modbus host:port, reload/upload controls
components/

Register Table
Live values, flash on change. Inline value edit, metadata edit, add row, delete
components/

Change Log
Scrolling feed. Source IP, old→new delta, timestamp
components/

HOOKS

useWebSocket
Auto-reconnect, exponential backoff, message dispatch

useRegisters
Snapshot merge, WS delta apply, CRUD methods (create / update / meta / delete)

:3000 | /ws

KEY DATA FLOWS

- CSV → RegisterStore — load on boot / hot-reload
- Modbus Master → DataBlock.setValues() — FC5/6/15/16 write
- DataBlock → EventBus — run_coroutine_threadsafe
- Dashboard → PATCH /registers/{addr} — user value edit
- EventBus → WebSocket broadcast — fan-out to all clients
- WebSocket → useRegisters — snapshot + delta merge

`POST /config/upload` → `RegisterStore.load()` live register swap